



Redes: TA048

TP1: File Transfer

04/10/2026 - Grupo 13

Integrantes:

Abel Tomas Modesti Funes	111932
Greco Lorenzo Donato	110118
Camilo Ignacio Campos Durán	109368
Nahuel Persia	90772
Emanuel Nicolas Sagman	103439

1. Introducción

En este informe se mostrará y describirá el trabajo realizado sobre el diseño e implementación de protocolos de transporte montados en la capa de aplicación con la intención de investigar y entender cómo es el mecanismo de transferencia de datos entre los diferentes métodos desarrollados.

Para lograr dicho objetivo se programaron dos protocolos de la capa de transporte, desde la capa de aplicación y utilizando UDP como mecanismo de envío de los paquetes. Esta decisión se toma, ya que UDP es un protocolo mínimo, con lo cual nos permite desde el lado aplicativo tener mayor control sobre el comportamiento y lograr características complejas al momento del intercambio de datos.

Ya que UDP es un protocolo que no se encarga de saber si el paquete llega al otro extremo, ni la forma en la que llegan (ordenas, duplicados, etc) y que el objetivo del trabajo era realizar protocolos RDT (Reliable Data Transfer) nos llevó a implementar los mecanismos necesarios para cumplir con las condiciones mínimas de una comunicación confiable.

Los protocolos desarrollados en cuestión son Stop & Wait (o SW, como lo podremos encontrar a lo largo del informe) y Selective ACK (o SACK).

Para lograr el objetivo de transferencia de datos se desarrolló, casi como una excusa, un pequeño software que puede enviar y recibir archivos entre cliente y servidor, el cual implementa los protocolos desarrollados en este trabajo.

En concreto, esta aplicación está dividida en tres programas principales:

- un servidor capaz de alojar archivos y recibir diferentes tipos de peticiones
- un cliente que puede enviar archivos “upload”
- un cliente que puede descargar archivos “download”

Cabe aclarar que este trabajo no solo constó en enviar archivos de un software a otro, sino que se desarrolló el header utilizado y que las velocidades de transferencia de archivos sean coherentes con aplicaciones utilizadas comúnmente en el ámbito del desarrollo de software.

2. Hipótesis y suposiciones realizadas

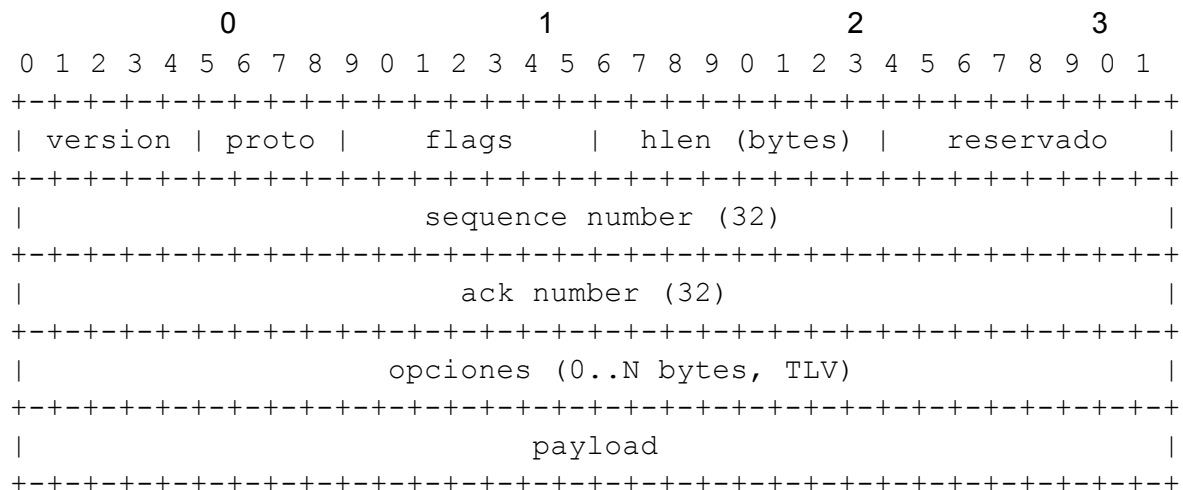
En este informe se toman las siguientes hipótesis y suposiciones:

- Se toma el modelo de 5 capas explicado a lo largo de la materia (aplicación, transporte, red, link, física).
- Los archivos que se enviarán con el protocolo deberán ser guardados en memoria, sin utilizar un buffer en disco. De esta manera se simplifica la operatoria no relacionada con un protocolo RDT.
- El límite máximo de tamaño de archivo es 1GB.
- Los paquetes enviados por UDP se llaman segmentos en este informe, al igual que TCP, para evitar confusiones con los datagramas de la capa de red.
- No se aseguran tiempos de transferencia para el protocolo SW.
- No se aseguran tiempos de transferencia para topologías más complejas que tener un servidor y un cliente conectados directamente con los parámetros pedidos en el enunciado del trabajo práctico.
- El archivo a enviarse o recibirse no tiene una extensión definida, con lo cual se podría enviar de cualquier tipo.
- Todas las pruebas deberán ser realizadas dentro de una misma computadora, utilizando mininet para emular redes, pero no se contempla el uso de dos host físicos en ningún momento de la prueba del presente trabajo.
- El MTT utilizado es 1420 para el desarrollo de todo el trabajo.
- Para hacer la prueba en vivo descargando el archivo directamente del campus de la facultad es condición indispensable que la notebook a utilizar pueda conectarse a la red FIUBA y que la misma preste las condiciones mínimas para realizar la descarga.
- La PC a utilizar para hacer pruebas debe tener al menos 4GB ram.
- El cliente debe tener conocimiento de la IP y puerto del servidor.

3. Implementación

3.1. Estructura de paquete RDT

El protocolo utiliza un encabezado estructurado en palabras de 32 bits (4 bytes) de ancho. Consta de un header fijo de 12 bytes, seguido por un campo de opciones de tamaño variable y el payload correspondiente.



flags: SYN | FIN | ERR | ACK | CAN | x | x | x

header fijo = 12 bytes; hlen = 12 + largo de las opciones

Versión: identifica la versión del protocolo que se está utilizando para enviar este paquete, se utiliza para garantizar compatibilidad entre el cliente y el servidor.

Proto: identifica el protocolo que se va a utilizar para la transferencia de datos (que se indicarán cuales son válidos más adelante).

Flags: Contiene flags de control de la conexión.

- SYN: Se utiliza para establecer la conexión entre el cliente y el servidor.
- FIN: Se utiliza para finalizar la conexión entre el cliente y el servidor.
- ERR: Flag utilizado para indicar posibles errores por parte del cliente o del servidor.
- ACK: Indica que el campo ack number es válido y confirma la recepción de datos.
- CAN: Se utiliza para cancelar la transaccion del archivo en mitad de la transaccion
- Flags reservados: se encuentran reservados para una posible implementación futura.

Hlen: marca el largo del header, en escala de bytes.

Reservado: Un espacio de bits reservado para implementación futura.

Sequence number (seq): marca el número de secuencia se utiliza para ordenar paquetes entrantes y detectar duplicados o fallas.

Acknowledge number (ackn): Número de acuse de recibo. Indica el número de secuencia que el receptor espera recibir o ha confirmado acumulativamente.

Opciones: este apartado en el presente trabajo práctico, en particular, sólo se utiliza para el protocolo SACK. A continuación mostramos la estructura del campo opciones:

```
+-----+
|  type  |
+-----+
| length |
+-----+
|  left  |
+-----+
|  right |
+-----+
```

- Type (1 Byte): Determina la opción. Vale 0x01 (en nuestro caso) y distingue la opción SACK
- Length (1 Byte): Longitud total del campo opciones, podemos saber cuántos bloques estarán, realizando la operación “ $2 \text{ (type + Length)} + 8(\text{Left} + \text{Right}) * N$ ”, donde N es la cantidad de bloques.
- Left (4 Bytes): Borde izquierdo del N-ésimo bloque, indica el primer número de secuencia del segmento abarcado.
- Right (4 Bytes): Borde derecho del N-ésimo bloque, indica el siguiente número de secuencia al último recibido.

Nota: conceptualmente left y right interpretan una tupla de principio/fin de SACK, donde juntos forman lo que denominamos **Bloques** (Intervalos) ordenados no mergeados, indicando la falta de paquetes faltantes intermedios.

Limitamos la formación máxima de bloques que se pueden armar a **8**, 1 bloque = [left, right), el emisor mantiene **rwnd = 8** segmentos en vuelo, por lo tanto nunca puede haber más de 3 bloques en circulación simultánea, y el valor elegido 4 deja un bloque de margen.

Payload: son los datos que queremos transmitir (para esta aplicación son una parte de un archivo).

Como nota de color, el header no utiliza un checksum de manera explícita, sino que nos apoyamos en el que utiliza UDP que es el protocolo de transporte que utilizamos por debajo para garantizar la integridad de los datos enviados.

3.2. Handshake

Se implementó un **three way handshake**, similar al que implementa el protocolo **TCP** (Transmission Control Protocol), el cual detallamos a continuación:

1. El cliente comienza la conexión enviando un paquete inicial con el flag especial **SYN** (Synchronize) encendido, el número de secuencia generado (el número es aleatorio cuando se genera y luego es acumulativo) y el protocolo de comunicación que se quiere utilizar (SW y SACK).
2. El servidor valida únicamente que el protocolo enviado sea válido y la versión de ambos host que realizarán la comunicación sea la misma, al ser validado, envía un paquete también con el flag de SYN activado, pero también con el de ACK, el número de secuencia de su lado y ACK number que se va a utilizar.
3. Al recibir esta información del lado cliente, actualiza el ACK y envía un segundo paquete al servidor con el flag de SYN no encendido, pero con el de ACK encendido.

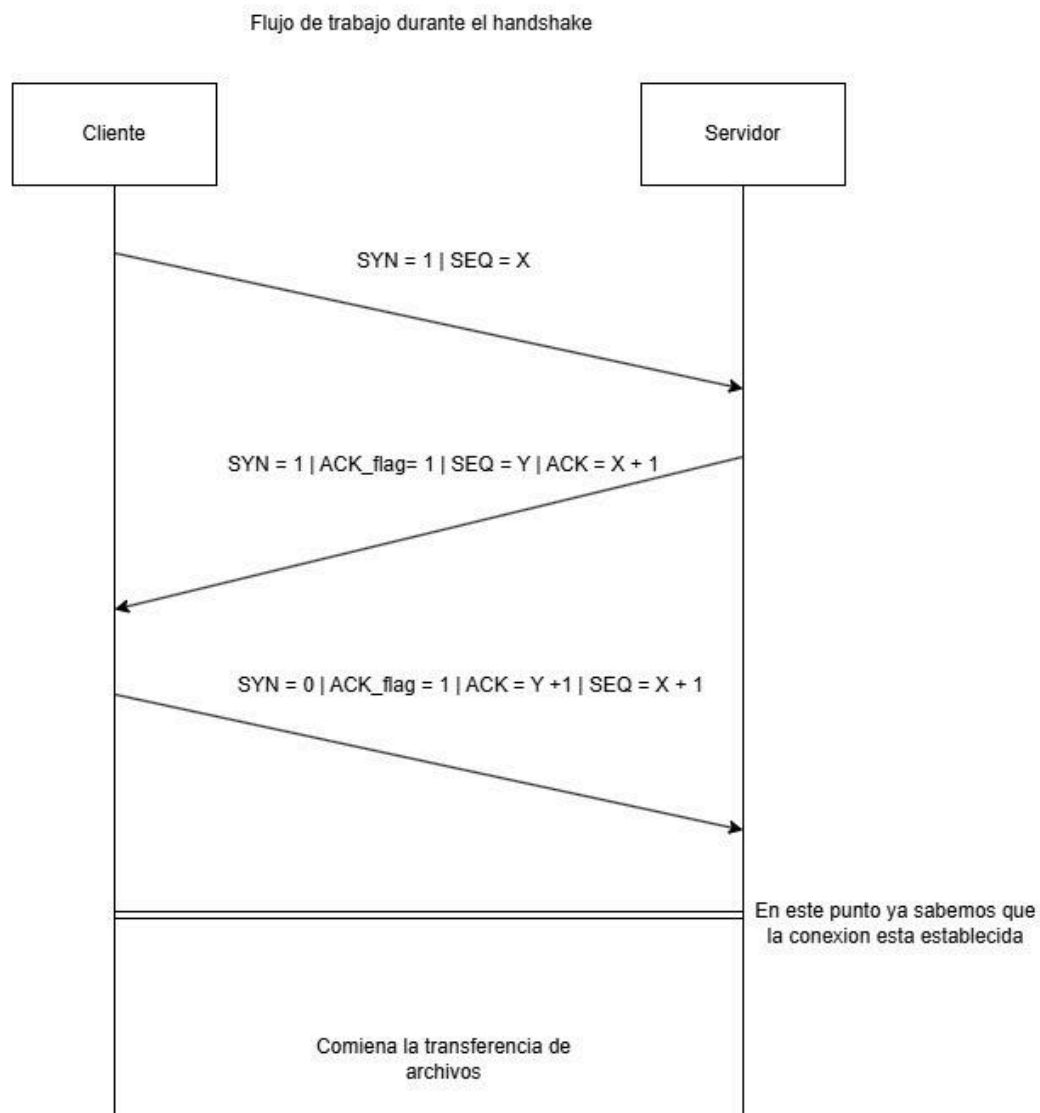


Figura 1: Flujo de trabajo de cliente y servidor durante el handshake.

3.3. Stop and wait

Antes de iniciar con la explicación específica del protocolo, vamos a hacer una pequeña aclaración. Aquí perdemos la notación de cliente y servidor para los hosts, sino que lo reemplazamos por **Emisor** y **Receptor**, ya que ambos hosts tienen la capacidad de enviar y recibir archivos.

1. El emisor envía un paquete y comienza la espera del ACK de respuesta del receptor. En este momento inicia un timeout.
2. En caso de alcanzarse el timeout, el emisor reenvía el paquete.
3. Nunca se va a avanzar hasta que el paquete que se está intentando enviar llegue a destino y el ACK correspondiente llegue al emisor.
4. En caso de intentar reenviarse una cantidad de veces definidas, 10, (en nuestro caso) sin lograr recibir el ACK correspondiente la comunicación se da por fallida y se cierra.
5. Si llega el ACK correspondiente por parte del receptor, el emisor procede a enviar el siguiente paquete, repitiendo el flujo hasta completar la transferencia.

Nota: como detalle de implementación adicional al protocolo, el servidor tiene timeouts aparte con el propósito de no dejar comunicaciones establecidas indefinidamente.

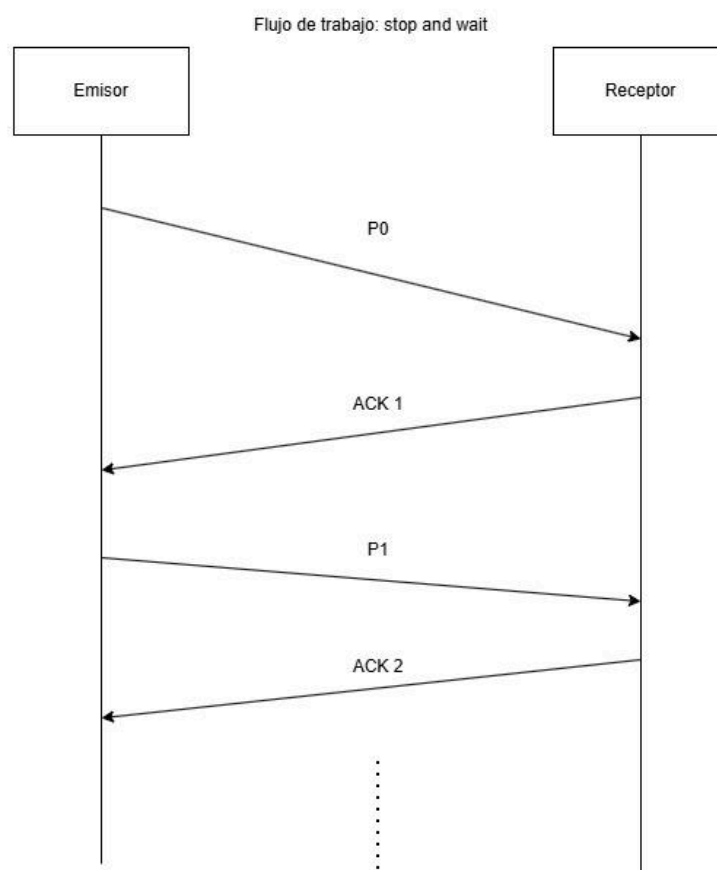


Figura 2: Flujo de trabajo durante la transferencia de archivo utilizando stop and wait.

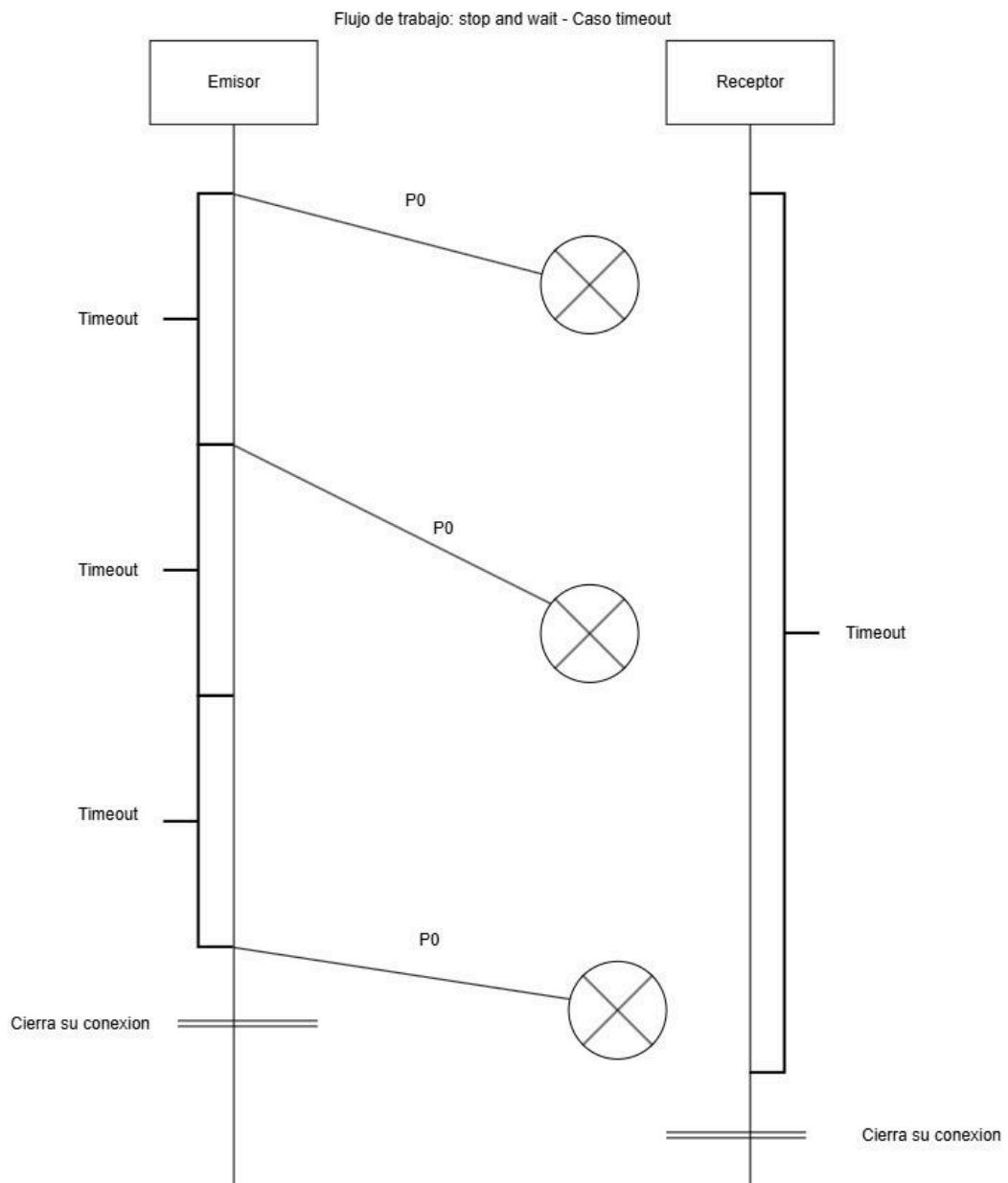
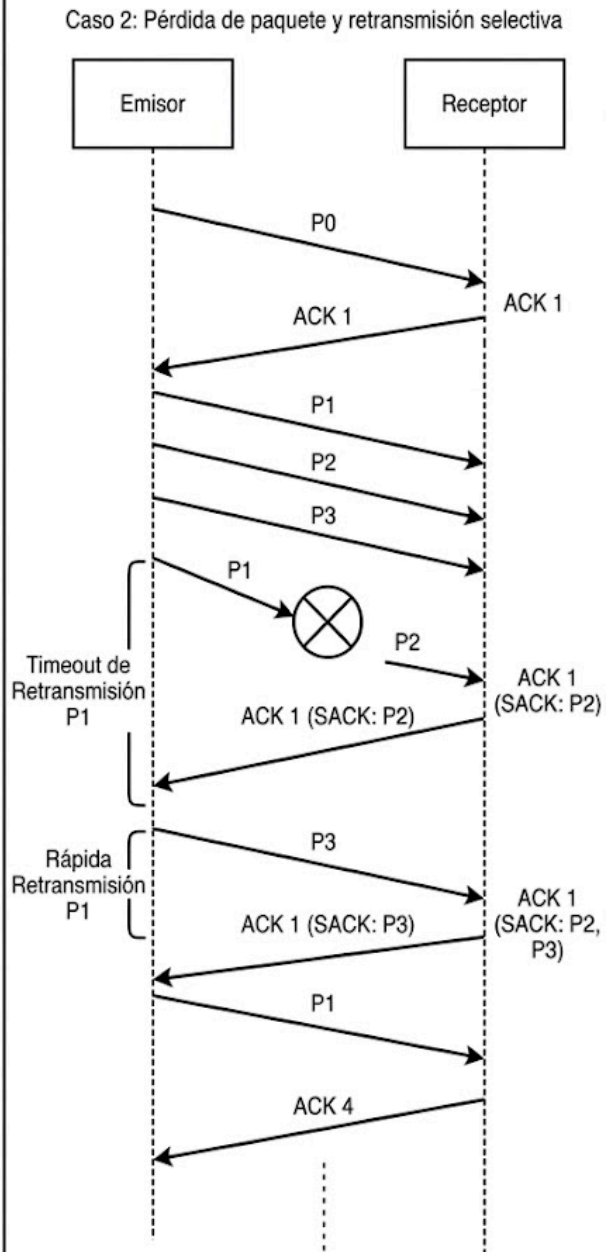
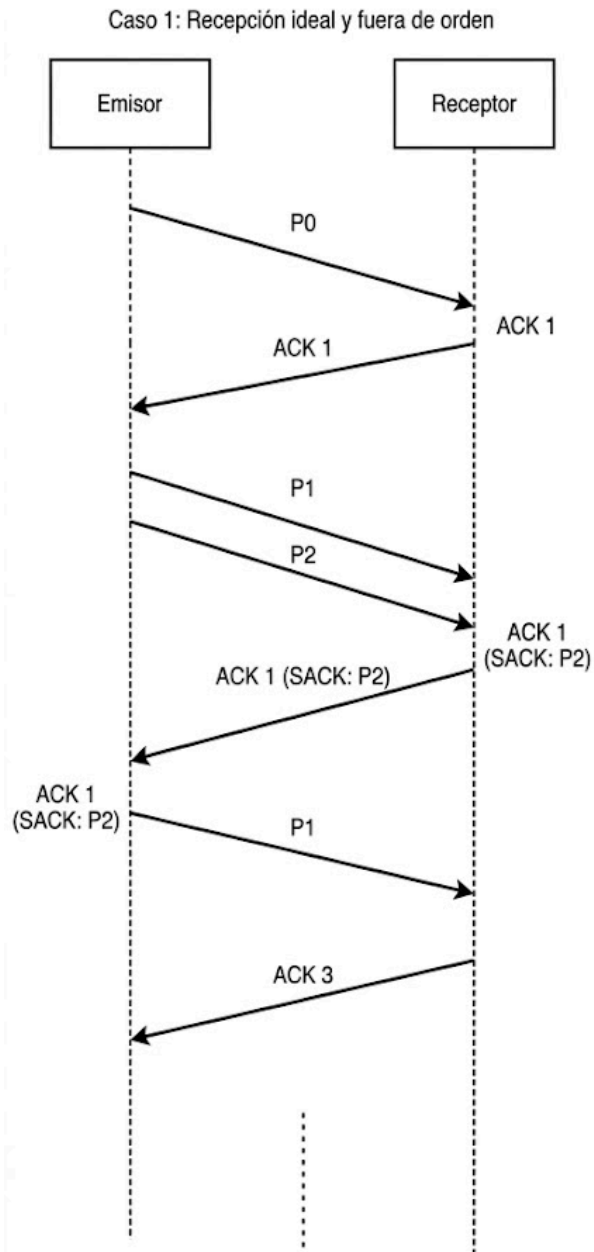


Figura 3: Cierre de conexión por timeout utilizando stop and wait.

3.4. SACK

1. **Envío en ráfaga / ventana:** El emisor envía varios paquetes seguidos (por ejemplo: P0, P1, P2, P3) sin detenerse a esperar el ACK de cada uno para mandar el siguiente.
2. **Control de retransmisión:** Por cada paquete enviado se inicia un temporizador (*timeout*).
3. **Confirmación selectiva ante pérdidas:** Si un paquete se pierde en el camino (por ejemplo, P1), pero los siguientes llegan bien (P2 y P3), el receptor guarda en memoria (bufer) los paquetes desordenados y responde con un mensaje especial:
 - **ACK acumulativo:** Indica el último paquete recibido correctamente en orden (ACK 1).
 - **Bloque SACK:** Notifica explícitamente los paquetes que sí llegaron fuera de orden (SACK: P2, P3).
4. **Retransmisión únicamente del faltante:** El emisor analiza la información de SACK y **retransmite exclusivamente el paquete faltante** (P1), sin necesidad de volver a enviar P2 ni P3.
5. **Avance de la ventana:** Una vez que el paquete faltante (P1) llega a destino, el receptor procesa todo el bloque completo y envía un nuevo ACK (ACK 4) que permite al emisor avanzar la ventana y continuar enviando los siguientes paquetes (P4,P5, etc.).
6. **Cierre por fallos (Límite de retransmisiones):** En caso de intentar reenviar un paquete faltante una cantidad definida de veces (por ejemplo, 10 intentos por *timeout*) sin lograr recibir su ACK o la actualización en la lista SACK, la comunicación se da por fallida y la conexión se cierra.

Flujo de trabajo: Selective ACK (SACK)



3.5. Finalizar conexión

Cuando la transferencia del archivo finaliza, comienza el cierre de la conexión.

En la programación de los protocolos, no fue especificado si cliente o servidor inicia el cierre. Ambos podrían hacerlo.

La aclaración es pertinente para la implementación de este trabajo en particular, ya que siempre el cliente es quien lo inicia.

Dicho esto, vamos a la explicación puntual del flujo de mensajes:

1. El cliente envía el flag FIN en 1 al servidor.
2. El servidor responde con el flag de ACK en 1
3. El servidor envía el flag de FIN en 1
4. El cliente responde con el flag de ACK en 1

Esto resulta en cierres de cada una de las partes, donde que se esté en periodo de cierre, de un lado, no significa que el otro no pueda mandar mas paquetes. Es más, lo hará hasta terminar tu tarea.

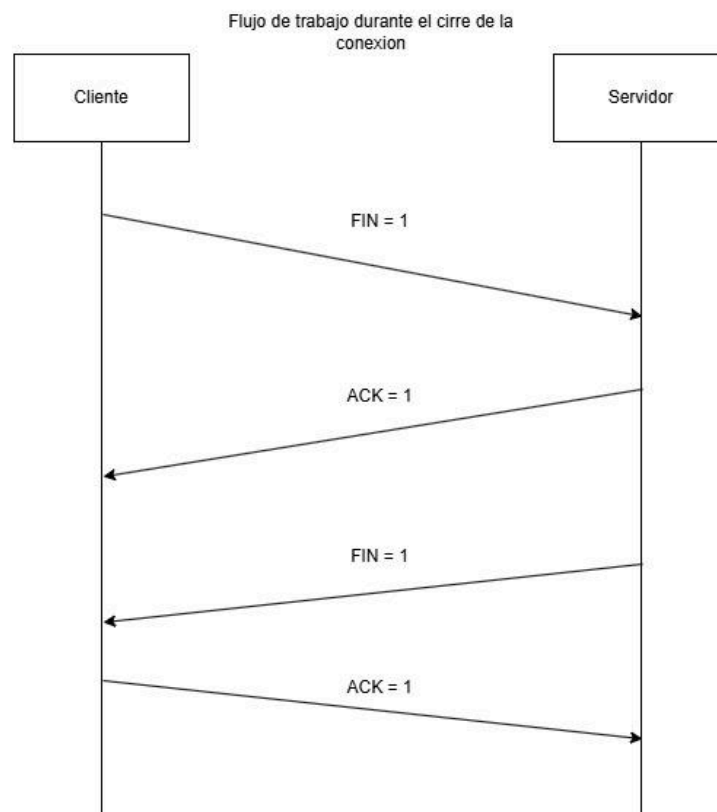


Figura 4: Flujo de trabajo del cliente y el servidor durante el cierre de la conexión.

4. Pruebas

A continuación detallamos las pruebas realizadas, las cuales van en línea con lo pedido tanto en el enunciado de trabajo práctico, como en los diferentes mensajes enviados por los profesores.

Dicho esto, no son las únicas pruebas que realizamos, ya que la idea era realizar un trabajo robusto que no cumpla lo mínimo pedido.

1. **Configuración de la topología:** levantar la topología con una pérdida de paquetes del **10 %** en cada sentido de la comunicación y un RTT de **40 ms**.

Para este escenario se desarrolló una tipología mínima, en la que contamos con dos hosts, cliente y servidor, los cuales se comunican en forma directa, utilizando mininet para llevar la red a algo un poco más realista y no la inmediatez de tener dos servicios corriendo en el mismo host.

Se configuraron los parámetros pedidos y estos se convirtieron en la base de todas nuestras pruebas.

2. **Verificación de la conectividad:** comprobar, mediante **ping**, que la topología cumple con los parámetros de red requeridos.

Utilizando mininet se iniciaron los dos hosts y se realizó ping desde el cliente al servidor para comprobar que se pueda llegar de uno a otro.

3. **Transferencia de un archivo de 5 MB:** realizar una transferencia utilizando el protocolo con **pipelining**. La transferencia deberá completarse en **menos de 2 minutos**.

Para este punto nos concentramos en el envío y recepción de archivos de pruebas de exactamente 5 MB y cronometrando el comportamiento del software. Según varias pruebas realizadas utilizando SACK, el tiempo promedio tanto para carga como descarga de archivos ronda los 57 segundos, cumpliendo con un RTT de 40ms y 10% de pérdida.

4. **Prueba de concurrencia:** demostrar que la aplicación funciona correctamente con múltiples clientes operando en simultáneo. Para ello, se deberán ejecutar en paralelo dos uploads y dos downloads desde diferentes hosts clientes.

Se probó que la aplicación puede atender a al menos dos clientes enviando archivos y dos recibiendo al mismo tiempo. Claramente esto afecta los tiempos de transmisión si lo comparamos con el caso ideal de un cliente y un servidor.

5. Preguntas teóricas a resolver

5.1. Describa la arquitectura cliente-servidor

La arquitectura de cliente servidor es un modelo de diseño de software, la encontramos en la capa de aplicación. Donde tenemos dos hosts que funcionan repartiéndose las tareas, uno es el **cliente**, que suele iniciar la comunicación y pedirle datos a un **servidor** (ambos son hosts que trabajan en capa de aplicación).

En este modelo el cliente es que por lo general comienza la comunicación a un determinado servidor con una **IP** o **hostname** fijo, los clientes no se comunican entre sí (eso sería un modelo de P2P). El proceso se reparte entre el cliente y el servidor.

El ciclo de vida del servidor siempre es mayor a la del cliente ya que debe estar activo para responder a las peticiones de diferentes clientes que quieran realizar peticiones.

El servidor puede no estar ejecutándose únicamente en un solo host, sino estar repartido de manera distribuida.

Ejemplos de aplicaciones que utilizan este modelo son el correo electrónico, plataformas de streaming, banca en línea, entre muchos otros. Este modelo de diseño es muy utilizado en la actualidad para todo tipo de aplicaciones en la red.

5.2. ¿Cuál es la función de un protocolo de capa de aplicación?

Un **protocolo de capa de aplicación** define reglas (como todo protocolo de otra capa, como capa red o capa transporte) de comunicación entre procesos de una aplicación distribuida, es decir, que funcionan en **hosts** (dos computadoras, o una computadora y un servidor, dos servidores, etc.) diferentes distribuidos por la gran red conocida como **internet**. Especifica qué mensajes se van a comunicar entre ambos (en esta capa los paquetes se conocen como mensajes), el orden en el que se van a transmitir y recibir y cómo se va a reaccionar a ellos.

Los protocolos de esta capa aprovechan el modelo de capas para abstraerse de todo lo relacionado de la comunicación a través de la red, capa de transporte, capa red, capa de conexión y capa física. De esta manera internet funciona como una infraestructura que permite el transporte de mensajes, mientras que el protocolo de aplicación establece las reglas para entenderse utilizando estos mensajes.

5.3. Detalle el protocolo de aplicación desarrollado en este trabajo

El protocolo de aplicación desarrollado que este informe reporta es un file transfer con las características RD, es decir un protocolo confiable. Por confiable entendemos que los datos enviados llegan al destino, no se corrompieron en el camino por alguna interferencia externa y no llegan duplicados ni desordenados. De esta manera podemos enviar archivos binarios de manera confiable.

5.4. La capa de transporte del stack TCP/IP ofrece dos protocolos TCP y UDP

¿Qué servicios ofrecen ambos protocolos?

A nivel capa de transporte, ambos protocolos comparten una responsabilidad: permitir la comunicación proceso a proceso mediante la multiplexación utilizando puertos, junto a una verificación básica de integridad del encabezado y datos mediante el checksum.

Sin embargo, difieren en el nivel de abstracción y los servicios adicionales que proveen:

Servicios de TCP:

- **Entrega confiable de datos:** Garantiza que todos los datos enviados llegue a destino sin errores, duplicados o pérdidas, utilizando un esquema de confirmaciones (ACKs), temporizadores (timers) y retransmisiones.
- **Entrega ordenada:** mantiene orden estricto de recepción de los paquetes utilizando números de secuencia, reordenando los paquetes si llegan en desorden.
- **Control de flujo:** evita que el emisor sature al receptor mediante el mecanismo de ventana deslizante, ajustando el ritmo de envío según la capacidad de procesamiento disponible en el receptor.
- **Control de congestión:** modula la tasa de transmisión del emisor en función del estado de la red para evitar el colapso del canal por sobrecarga.
- **Gestión de la conexión:** establece y finaliza un estado de conexión entre el emisor y receptor antes y después del intercambio de datos.

Servicios UDP:

- **Transporte de datagramas:** envía paquetes de manera independiente sin garantizar su entrega, orden o duplicación.
- **Baja latencia y sin retraso de establecimiento:** permite la transmisión de datos sin necesidad de realizar una fase previa de negociación (handshake).
- **Conservación de límites de mensaje:** mantiene las fronteras de los bloques de datos enviados por la aplicación.

¿Cuáles son sus características?

Característica	TCP	UDP
Orientación	Orientado a conexión (requiere estado explícito en ambos extremos).	Sin conexión (connectionless, no guarda estado).
Abstracción de datos	Flujo continuo de bytes (byte-stream sin fronteras de mensaje).	Datagramas discretos (datagram-oriented).
Confiabilidad	Alta (garantizada por ACK, SACK, retransmisión).	Nula en capa de transporte (depende de la capa de aplicación si se requiere).
Sobrecarga	Encabezado variable de 20 a	Encabezado fijo y liviano, de 8

	60 bytes; mayor uso de memoria y CPU para mantener estados y temporizadores.	bytes, mínimo consumo de recursos.
Control de flujo	Protege al receptor mediante ventana deslizante (rwnd).	Si el receptor se satura, los paquetes se pierden.
Control de Congestión	Protege a la red adaptando la velocidad (cwnd).	Transmite a la tasa que le imponga la aplicación.
Modo de transmisión	Exclusivamente Unicast (1 a 1)	Unicast, Multicast y Broadcast (1 a 1, 1 a N, 1 a Todos).

¿Cuándo es apropiado utilizar cada uno?

UDP (User Datagram Protocol), es un protocolo de **capa de transporte** muy utilizado por su mecanismo simple para enviar paquetes a través de la red (nombrados datagramas en algunos escritos pero en este informe se llamarán segmentos, al igual que TCP). Es un protocolo que no ofrece ningún mecanismo de confirmación de envío ni orden de llegada ausencia de duplicados y tampoco retransmisión ante pérdida, solamente envía un determinada cantidad de datos a una determinada dirección IP. Tampoco ofrece ningún mecanismo de control de flujo o congestión, solamente encapsula el paquete que llega de la capa de aplicación y lo envía por la red.

Teóricamente está implementado únicamente en los hosts, ya que sirven para enviar mensajes a la capa de aplicación (que en un modelo ideal se encuentra también únicamente en los hosts), aunque sabemos que en la realidad no ocurre.

Este protocolo ofrece un header reducido contando únicamente con los puertos origen y destino, el checksum y la longitud del segmento.

Debido a su simplicidad y su bajo overhead por su funcionamiento simple, es utilizado en aplicaciones donde se puede tolerar pérdidas de paquetes o donde la confiabilidad no es un requisito. Un ejemplo super conocido son los DNS, servicios de streaming.

TCP (Transmission Control Protocol), es un protocolo de **capa de transporte** diseñado para brindar un servicio de transmisión de datos sumamente confiable y ordenado entre procesos. A diferencia de UDP, es un protocolo orientado a conexión que requiere establecer un estado explícito en ambos extremos mediante una fase previa de negociación (como el *three-way handshake* implementado a nivel aplicativo en este trabajo).

Para garantizar la integridad de la información, TCP ofrece mecanismos de confirmación de recepción (ACKs), retransmisión automática ante pérdidas por temporizadores, ordenamiento de segmentos desordenados mediante números de secuencia, y algoritmos de control de flujo y congestión. Todo este funcionamiento implica mantener buffers de memoria y temporizadores activos en ambos hosts, sumado a un encabezado con mayor sobrecarga.

Debido a estas garantías de confiabilidad, es apropiado utilizar TCP en aplicaciones donde la integridad, el orden y la precisión de los datos sean indispensables, y donde no sea admisible la pérdida de información. El caso de uso por excelencia es la transferencia de archivos, así como también la navegación web, el correo electrónico y las terminales de acceso remoto (SSH), donde cada byte transmitido debe llegar intacto al receptor.

5.5. Justifique si el protocolo desarrollado cuenta con mecanismos de congestión, en caso de tenerlos, describirlos

Nuestro protocolo desarrollado no cuenta con mecanismos de congestión como lo hacen otros protocolos tomando como ejemplo TCP.

Consultamos y no es parte indispensable del propósito de este trabajo.

5.6. ¿Cuál de los dos protocolos desarrollados envía archivos en la menor cantidad de tiempo? ¿Es siempre el mismo?

En este trabajo práctico se desarrollaron dos protocolos contrapuestos en su filosofía, uno que envía el paquete y hasta que no se asegura que llegó, no continúa (SW), y otro que envía flujos de paquetes y reintentar solo con los que no fueron enviados correctamente (SACK).

- **Selective ACK:** SACK supera a Stop and Wait en cualquier red real o en internet porque **no pierde tiempo**: te tira un paquete atrás del otro en ráfaga y, si en el camino se le pierde uno solo, el receptor le avisa exactamente cuál falta para **retransmitir únicamente ese sin frenar el flujo**.

Básicamente, si querés velocidad, transferir archivos pesados o la conexión tiene bastante latencia, SACK le pasa el trapo a SW porque aprovecha todo el caño al máximo y no te deja la red colgada esperando que le confirmen cada pelotudez.

- **Stop-and-Wait (SW):** Stop and Wait solo gana cuando la velocidad es nula, manda solo paquete y se queda esperando que le devuelvan el ACK antes de mandar el siguiente, no necesita andar guardando nada en un búfer.

6. Dificultades encontradas

En este apartado se enumeran las principales dificultades técnicas enfrentadas durante el diseño e implementación de los protocolos RDT en la capa de aplicación sobre UDP:

1. Diseño optimizado del encabezado:

Encontrar el balance adecuado entre un overhead mínimo y la cantidad de información de control necesaria representó un desafío de diseño. Fue necesario estandarizar un encabezado fijo de 12 bytes y estructurar el campo de opciones mediante formato TLV (Type-Length-Value) para transportar los bloques SACK sin generar fragmentación innecesaria en la capa de transporte.

2. Gestión de temporizadores y retransmisiones en SACK:

A diferencia de Stop & Wait, donde la gestión del temporizador es secuencial, en SACK la transmisión en ráfaga requirió controlar temporizadores o estados de expiración por cada segmento en vuelo. Ajustar el valor de timeout para evitar retransmisiones espurias (falsos positivos) ante la latencia emulada de 40 ms y la pérdida del 10% de paquetes requirió un ajuste refinado en la lógica del emisor.

3. Manejo de buffers y reordenamiento de bloques fuera de orden:

En el receptor, la implementación de la lógica para almacenar segmentos fuera de orden y mantener actualizada la lista de intervalos SACK (tuplas [left, right)) implicó una complejidad algorítmica considerable. Fue necesario garantizar la fusión y actualización dinámica de estos bloques a medida que llegaban los paquetes faltantes, evitando la entrega de datos desordenados a la capa de aplicación.

4. Sincronización y concurrencia de múltiples clientes:

Garantizar que el servidor atendiera simultáneamente dos peticiones de upload y dos de download sin bloquear la recepción de paquetes ni entrecruzar los estados de las sesiones requirió una arquitectura multihilo/asíncrona cuidadosa.

5. Cierre ordenado de la conexión (Handshake de finalización):

El diseño del flujo de finalización (intercambio de FIN/ACK) presentó casos de borde cuando los paquetes de cierre se perdían en la red emulada. Garantizar que ambos extremos liberaran los recursos y cerraran sus sockets sin quedar colgados indefinidamente exigió implementar timeouts específicos para la fase de cierre.

7. Conclusión

En el presente trabajo práctico se logró diseñar e implementar de manera satisfactoria un sistema de transferencia de archivos sobre la capa de aplicación, utilizando el protocolo de transporte UDP como base para la construcción de dos protocolos confiables: Stop & Wait (SW) y Selective ACK (SACK).

Mediante la emulación de red en Mininet con condiciones adversas (10% de pérdida de paquetes y 40 ms de RTT), se verificó experimentalmente la eficacia de los mecanismos de confiabilidad implementados (handshake de 3 vías, temporizadores por timeout, números de secuencia y confirmaciones acumulativas y selectivas).

Los resultados obtenidos permitieron contrastar el desempeño de ambos protocolos:

1. Stop & Wait demostró ser una solución conceptualmente simple y liviana en uso de memoria al no requerir buffers complejos; sin embargo, su rendimiento se ve fuertemente penalizado por el tiempo de ida y vuelta (RTT), ya que mantiene el canal ocioso a la espera de cada confirmación individual.
2. Selective ACK (SACK) optimizó sustancialmente el uso del ancho de banda y la velocidad de transferencia, logrando completar la transmisión del archivo de prueba de 5 MB en aproximadamente 57 segundos (cumpliendo holgadamente el requisito de 2 minutos). Esto se debe a su esquema de transmisión en ráfaga/ventana y a la capacidad del receptor de notificar bloques fuera de orden, retransmitiendo únicamente los segmentos perdidos sin detener el flujo global.

En conclusión, la realización de este trabajo brindó una comprensión profunda sobre los desafíos del diseño de protocolos de transporte confiables. La experiencia práctica evidenció que la incorporación de técnicas de pipelining y una adecuada gestión de buffers en memoria resultan indispensables para optimizar la tasa de transferencia (throughput) y garantizar la integridad de los datos en escenarios con latencia y pérdida de paquetes.